

Multi-container Applications with Compose



Compose Overview

- Podman Compose is open source.
- Many applications developed as microservices
- Manage multi-container applications
- Secure and lightweight alternative to Docker Compose
- Support for rootless containers
- Excellent choice for both development and production environments
- Uses YAML definition

Compose case use

```
# podman run --name www -d httpd
# podman logs w1
# podman stop w1
# podman start w1
# podman pause w1
# podman resume w1
# podman rm -f w1A
```

```
# podman run --name db -d mysql
# podman logs db1
# podman stop db1
# podman start db1
# podman pause db1
# podman resume db1
# podman rm -f db1
```

```
# podman run --name jv -d java
# podman logs jv
# podman stop jv
# podman start jv
# podman pause jv
# podman resume jv
# podman rm -f jv
```

```
# podman-compose up -d
# podman-compose ps
# podman-compose logs
# podman-compose down
# podman-compose pause
# podman-compose resume
# podman-compose exec
```

The Compose File

The Compose file is a YAML file that contains the following sections:

- **version** (deprecated): Specifies the Compose version used.
- **services**: Defines the containers used.
- **networks**: Defines the networks used by the containers.
- **volumes**: Specifies the volumes used by the containers.
- **configs**: Specifies the configurations used by the containers.
- **secrets**: Defines the secrets used by the containers.

The secrets and configs objects are mounted as a file in the containers.

Simple compose YAML definition – example 1

```
# vi compose.yaml
services:
  webserver1:
    container_name: web1
    image: quay.io/jason_wong76/webserver:latest
    ports:
      - 8080:80
  webserver2:
    container_name: web2
    image: quay.io/jason_wong76/webserver:latest
    ports:
      - 8181:80
```

Simple compose YAML definition – example 2

```
# vi compose.yaml
services:
  backend:
    image: quay.io/example/backend
    ports:
      - "8081:8080"
    command: sh -c "COMMAND"
  db:
    image: registry.redhat.io/rhel8/postgresql-13
    environment:
      POSTGRESQL_ADMIN_PASSWORD: redhat
```

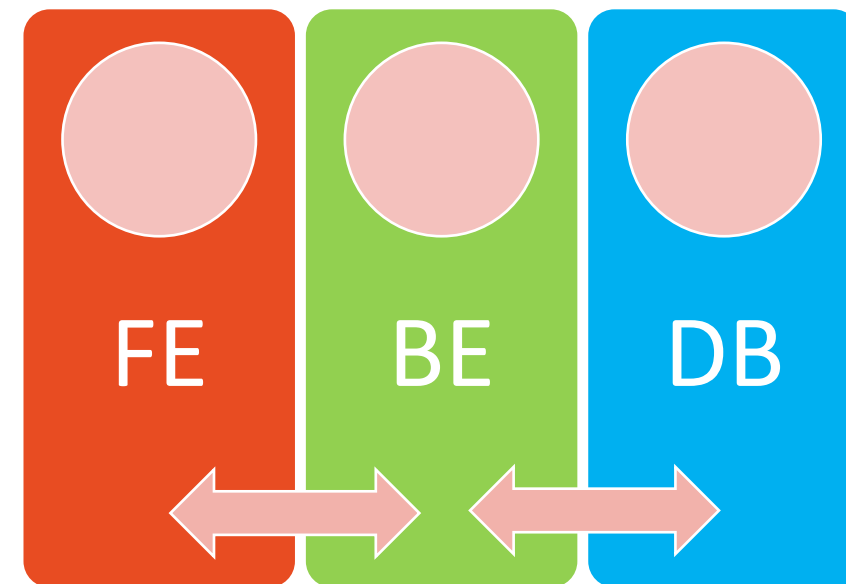
Networking

```
services:
  frontend:
    image: quay.io/example/frontend
    networks: ❶
    - app-net
    ports:
    - "8082:8080"
  backend:
    image: quay.io/example/backend
    networks: ❷
    - app-net
    - db-net
  db:
    image: registry.redhat.io/rhel8/postgresql-13
    environment:
      POSTGRESQL_ADMIN_PASSWORD: redhat
    networks: ❸
    - db-net

networks: ❹
  app-net: {}
  db-net: {}
```

2 networks: app-net and db-net

3 containers: frontend, backend and db



Volumes

```
services:
  db:
    image: registry.redhat.io/rhel8/postgresql-13
    environment:
      POSTGRESQL_ADMIN_PASSWORD: redhat
    ports:
      - "5432:5432"
    volumes: ❶
      - db-vol:/var/lib/postgresql/data

volumes: ❷
  db-vol: {}
```

- Use **volumes** keyword
- Mount **db-vol** to **/var/lib/postgresql/data** in the container

Mount volume created by *podman volume create*

```
services:
  db:
    image: registry.redhat.io/rhel8/postgresql-13
    environment:
      POSTGRESQL_ADMIN_PASSWORD: redhat
    ports:
      - "5432:5432"
    volumes:
      - my-volume:/var/lib/postgresql/data

volumes:
  my-volume:
    external: true
```

- Create external volume
podman volume create my-volume
- Use **volumes** keyword point to external volume

Bind volume by providing path on host machine

```
services:
  db:
    image: registry.redhat.io/rhel8/postgresql-13
    environment:
      POSTGRESQL_ADMIN_PASSWORD: redhat
    ports:
      - "5432:5432"
    volumes:
      - ./local/redhat:/var/lib/postgresql/data:Z
```

- Podman mounts **./local/redhat** directory on host machine to **/var/lib/postgresql/data** in container
- SELinux is set properly

Install podman-compose

- On Linux Redhat family systems :

```
# pip3 install --user podman-compose
```

- On Linux Debian/Ubuntu family systems :

```
# apt -y update && apt install -y podman-compose
```

- Check versions

```
# podman-compose version
```

- Get help

```
# podman-compose help
```

compose YAML definition

```
# vi compose.yaml
```

```
services:
```

```
  webserver1:
```

```
    container_name: web1
```

```
    image: quay.io/jason_wong76/webserver:latest
```

```
    ports:
```

```
      - 8080:80
```

```
  webserver2:
```

```
    container_name: web2
```

```
    image: quay.io/jason_wong76/webserver:latest
```

```
    ports:
```

```
      - 8181:80
```

Usages of podman compose

Start the web applications. Make sure you are in same folder as the YAML file.

```
# podman-compose up -d
```

Confirm the web applications are started, and query web contents.

```
# podman-compose ps
```

```
# curl localhost:8080
```

```
# curl localhost:8181
```

View logs for the all services defined in YAML file.

```
# podman-compose logs
```

Other Lifecycle using podman compose

- Stop all services
podman-compose stop
- Stop specific service
podman-compose stop webserver1
- Start all services
podman-compose start
- Start specific service
podman-compose start webserver2
- Pause and resume services
podman-compose pause
podman-compose unpause

Other Lifecycle using podman compose

- Get interactive shell of container

```
# podman-compose exec webserver1 sh
uname -a
curl localhost
date
exit
```

- Execute commands non-interactively of container

```
# podman-compose exec webserver2 uname -a
# podman-compose exec webserver2 date
# podman-compose exec webserver2 ls -l
# podman-compose exec webserver2 curl localhost
```


Exercise 1 : More advanced compose file

- Add three webserver with respective configuration
 - Service name: webserver1
 - Container-name: web1
 - Image: quay.io/jason_wong76/webserver:latest
 - Forward port: 8080:80
- Service name: webserver2
- Container-name: web2
- Image: quay.io/jason_wong76/webserver:latest
- Forward port: 8181:80

Exercise 1 : More advanced compose file

- Add three web servers with respective configuration
 - Service name: webserver3
 - Container-name: web3
 - Image: quay.io/jason_wong76/webserver:latest
 - Forward port: 8282:80

Exercise 1 : More advanced compose file

- Add two database servers with respective configuration
 - Service name: dbserver1
 - Container-name: db1
 - Image: docker.io/library/mysql
 - Forward port: 13306:80
 - Environment variables:
 - MYSQL_ROOT_PASSWORD: cangetin
 - MYSQL_USER: dba
 - MYSQL_PASSWORD: cangetin
 - MYSQL_DATABASE: datadb1

Exercise 1 : More advanced compose file

- Add two database servers with respective configuration
 - Service name: dbserver2
 - Container-name: db2
 - Image: registry.access.redhat.com/rhscl/mysql-80-rhel7
 - Forward port: 23306:80
 - Environment variables:
 - MYSQL_ROOT_PASSWORD: cangetin
 - MYSQL_USER: dba
 - MYSQL_PASSWORD: cangetin
 - MYSQL_DATABASE: datadb2

Exercise 1 : More advanced compose file

Pull, start all containers using compose.yaml in detached mode. Make sure to create all containers in projectA project

```
# podman-compose -p projectA up -d
```

Verify all resources creation

```
# podman-compose -p projectA ps
```

```
# podman-compose -p projectA logs
```

```
# podman-compose -p projectA config
```

Test access to applications in containers

```
# curl -m 1 localhost:8080
```

```
# curl -m 1 localhost:8181
```

```
# curl -m 1 localhost:8282
```

```
# mysql -h127.0.0.1 -P13306 -p
```

```
<enter cangetin when prompted password>
```

```
mysql> show databases;           # should see databdb1
```

Exercise 2 : SQL + WORDPRESS

- Add a db server with respective configuration
 - Service name: database
 - Container-name: mydb
 - Image: [docker.io/library/mysql](https://hub.docker.com/r/library/mysql)
 - Environment variables:
 - MYSQL_ROOT_PASSWORD: redhat
 - MYSQL_USER: redhat
 - MYSQL_PASSWORD: redhat
 - MYSQL_DATABASE: datadb

Exercise 2 : SQL + WORDPRESS

- Add a wordpress server with respective configuration
 - Service name: main
 - Container-name: main
 - Image: docker.io/library/wordpress:latest
 - Port forwarding: 7070:80
 - Environment variables:
 - WORDPRESS_DB_PASSWORD: redhat
 - WORDPRESS_DB_USER : redhat
 - WORDPRESS_DB_NAME : datadb
 - WORDPRESS_DB_HOST : "mydb:3306"

Exercise 2 : SQL + WORDPRESS

Pull, start all containers using compose.yaml in detached mode. Make sure to create all containers in projectB project

```
# podman-compose -p projectB up -d
```

Verify all resources creation

```
# podman-compose -p projectB ps
```

```
# podman-compose -p projectB logs
```

```
# podman-compose -p projectB config
```

Test access to applications in containers

Launch browser and points to localhost:7070 / <container-host-ip>:7070

In this chapter, you learned:



The OpenShift web console provides a GUI for visualizing and managing OpenShift resources.



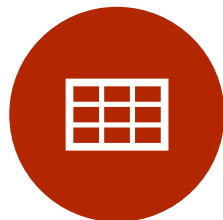
Some resources feature a specialized page that makes creating and editing resources more convenient than writing YAML by hand, such as the Edit Key/Value Secret editor, which automatically handles Base64 encoding and decoding.



You can install partner and community operators from the embedded OperatorHub page.



Cluster-wide metrics such as CPU, memory, and storage usage are displayed on the Dashboards page.



Project Details pages display metrics specific to the project, such as the top ten memory consumers by pod and the current resource quota usage.